

Union-Find

Disjoint Set Union, Connectivity · Mentor-Led Lab (Student Edition)

Developed by TalenciaGlobal

Difficulty ★★ Intermediate

Problem

Implement a union-find (disjoint set union) structure and process a sequence of union and query operations, answering each connectivity query and reporting the final number of groups.

This structure is the engine behind Kruskal's algorithm.

What to Compute

- For each query a b: 'query a b = Yes' if a and b are in the same group, otherwise No.
- Components: the number of distinct groups after all operations.

Input / Output Format

Input: Line 1: N T. Next T lines: 'union a b' or 'query a b'.

Output: One line per query, then Components.

Examples

Example 1

Input: 5 6 union 0 1 union 2 3 query 0 1 query 0 2 union 1 3 query 0 3

Output:

```
query 0 1 = Yes
query 0 2 = No
query 0 3 = Yes
Components: 2
```

Explanation: After unions, {0,1} then {2,3}; the final 'union 1 3' merges them, so 0 and 3 are connected; the groups are {0,1,2,3} and {4}.

Example 2

Input: 4 4 union 0 1 union 2 3 query 1 2 query 0 1

Output:

```
query 1 2 = No
query 0 1 = Yes
Components: 2
```

Explanation: Two separate pairs remain: {0,1} and {2,3}.

Constraints

- A union merges the groups of a and b.
- A query reports whether a and b are already connected.
- Components are counted as distinct set roots.

Sample Run

Sample Input

```
5 6
union 0 1
union 2 3
query 0 1
query 0 2
union 1 3
query 0 3
```

Sample Output

```
query 0 1 = Yes
query 0 2 = No
query 0 3 = Yes
Components: 2
```